

HelixCluster

HelixCluster

面向AI计算的分布式操作系统——从数据中心GPU到边缘手持设备，统一控制平面。

概要

Helix集群操作系统是新一代分布式操作系统，能够在异构节点上协调计算资源——从数据中心GPU到边缘单板计算机（SBC）及手持设备——将高性能计算调度、容器编排、AI/ML推理、联邦多集群操作及安全多租户会话整合于单一控制平面之下。

简要描述

基于Go的分布式操作系统 / GPU共享计算集群。它统一了高性能计算调度（采用Omega模型的双层调度器）、容器编排、AI推理路由、联邦管理及安全多租户会话，适用于异构节点，通过SWIM gossip协议和Raft共识协调运行，并采用后量子端到端加密技术。

详细描述

Helix集群操作系统能够在极度异构的硬件上协调计算负载——从数据中心GPU、边缘单板计算机，乃至手持设备——统一于单一控制平面之下，将一组A100 GPU机架与一批单板计算机视为可寻址的统一计算网络，而非十余个互不兼容的孤岛。该系统是一个基于Go的工作空间（包含单一代码库及Git子模块），实现了从L0硬件层到L7联邦与可观测性的七层架构，由十四个控制平面微服务协调运行。节点成员关系通过SWIM gossip协议和发现机制进行跟踪，使得计算网络能够在节点加入或离开时自愈；强一致性状态则依赖Raft共识维护，采用分片式Raft组结构，通过租约持有者本地读取提升速度，并借助STONITH隔离机制确保被分区的节点无法破坏共享状态。

工作负载调度采用Omega模型的双层调度器——支持乐观并发、ClassAd匹配、成组调度、基于价值乘数的抢占及约束性放置。不仅如此，它还超越了传统高性能计算调度器的范畴：支持碳排放感知与成本/TCO感知路由、云爆发自动扩展，并集成市场适配器（如Akash、io.net、RunPod、AWS Spot、Chutes），在本地资源不足时将作业溢出至租用容量。

最终用户无需直接接触这些底层机制，他们通过简洁的会话模型（计算资源分配）、交互式WebSocket/PTY终端、内部AI推理路由及资源池利用率读取进行交互。安全性被视为核心层而非附加功能：采用SPIFFE身份认证、设备验证（挑战/响应、GPU工作量证明、密封机制）、出口管制KYC门禁，以及基于X25519 + ML-KEM-768混合密钥交换的后量子端到端加密传输，配合AEAD记录保护与重放攻击防御——确保即使在未来量子计算威胁下，当前捕获的流量依然保持机密性。正确性并非仅靠声明，而是通过验证实现：确定性模拟测试（类似FoundationDB的种子运行、故障注入、网络模拟、逐字节重放及Porcupine线性一致性检查器）能够按需重现分布式故障，而强制配对变异测试则证明防护测试真正有效。架构与文档的准确性由机械化检查工具确保，一旦现实与文档出现偏差，构建过程即刻失败。

内容

为何构建

为了在截然不同的硬件层级上运行AI与HPC工作负载，无需拼凑独立的调度器、编排器和推理栈——并确保每一项交付的功能均经过真实终端用户行为验证（绝不依赖绿色测试或桩代码），且每项平台特定能力均基于真实原生设施实现（不采用仅限Linux的模拟方案）。项目治理文档中引用的核心问题在于“测试通过，功能却无法实际运行”的失败模式，而该项目正是为彻底消除这一问题而生。

为何颠覆行业

它将通常分属五套独立系统的功能——HPC调度、容器编排、AI推理、多集群联邦及安全多租户会话——整合为单一控制平面，覆盖从数据中心GPU到边缘手持设备的全场景。且其严谨程度堪比专业基础设施：采用形式化方法级别的正确性保障（TLA+规约、确定性模拟、线性一致性检查）与后量子机密传输，这类保障是多数编排器不敢尝试的。技术优势之外，成本与碳排感知的调度策略，加上云市场弹性扩展，更使其成为经济杠杆——调度器能自动追踪更廉价、更绿色或闲置的资源，在无需重写作业的情况下，降低同一工作负载的成本与排放。

创新之处

- **确定性模拟测试（DST）**——基于种子的全可复现模拟器，可注入故障、时钟偏移及网络分区，逐字节重放并通过Porcupine线性一致性检查器验证，确保捕获的海森堡bug可随时复现，永不遗失。

- **Omega模型双层调度器**——基于乐观并发的调度机制，结合ClassAd匹配、群调度及价值乘数抢占，共享状态设计让多个调度器能并行提交任务至同一集群，无需中央瓶颈。
- **后量子端到端加密/机密推理链路**——采用X25519 + ML-KEM-768混合密钥交换，每次请求绑定独立响应密钥对，并配备AEAD抗重放机制（加密原语已通过验证，完整的多节点机密往返流程仍处于计划中/受控阶段）。
- **基于证明的信任机制**——节点需证明其身份：SPIFFE标识、GPU工作量证明、设备密封、出口管制KYC审核及欧盟AI法案合规文档生成，信任通过证据建立，而非依赖网络位置假设。
- **成本与碳排感知编排**——TCO建模、碳排感知调度、云爆发、N+K故障预留及云市场适配器，将价格与排放纳入调度首要考量，而非事后补救。
- **多Raft共识**——每个分片独立Raft组，结合租约持有者本地读取实现低延迟一致性，并通过STONITH隔离（IPMI/EC2/Azure/SBD）确保故障节点被果断移除，避免状态污染。
- **反漂移机械化检查**——archlint在文档组件与实际包路径不匹配时立即中断构建，文档链引擎确保Markdown/HTML/PDF/DOCX内容字节级一致，杜绝文档与代码悄然脱节。

最大技术挑战及解决方案

- 「假通过」 (**PASS-bluff**, 测试在非功能特性上通过)：整个项目旨在消除的致命模式——测试套件在桩代码上显示绿灯。解决方案是强制配对变异测试：每个工作项均附带一个命名守卫测试，必须在独立代码变异下失败后，该项才能标记为完成。如此，通过的测试能够证明其验证的是真实行为而非模拟。
- **跨平台一致性（无 Linux 专用模拟）**：通过构建标签将共享接口拆分至各操作系统原生设施——Linux 的 cgroup、/proc、内核 WireGuard；macOS 的 sysctl、vm_stat、IOKit、wireguard-go——并通过独立操作系统预言机进行交叉验证，确保每个平台报告真实的原生状态，而非 Linux 的虚构实现。
- **分布式故障下的正确性**：采用确定性模拟测试与线性化检查器，人为制造并重放网络分区、崩溃及时钟偏移，并由 TLA+ 形式化规约在代码运行前锁定共识与调度不变量。
- **文档与架构偏移**：通过 archlint 实现，若文档中映射的包不存在，构建将直接失败；同时设置 docs_chain 验证关卡，无例外机制——架构偏移即构建中断，而非过时的维基页面。
- **未完成工作的诚实范围界定**：多节点推理端到端流程刻意设置在工单后，并标注「尚末端到端验证」，而非包装成已交付成果。这种严谨同样适用于尚未完成的工作，一如已完成的部分。

技术栈

- **Go (go.mod: 1.25 / 工具链 1.26.4)**：覆盖约 30 个模块的控制平面语言，因其轻量级 goroutine 并发及静态二进制部署优势，从数据中心到边缘设备均可一致部署。
- **Zig (0.14+) + C/C++**：在 Go 运行时成为障碍的场景下使用，如底层系统原语及 GPU 内核，需确定性、无分配的硬件控制。

- **gRPC + Protocol Buffers**: 各子系统间的 API (api/v1/) 均为类型化、版本化契约, 使十四个微服务能独立演进, 无需手动处理线上格式。
- **Raft (etcd-raft) + SWIM Gossip**: 刻意分层设计——Raft 负责必须强一致的状态, SWIM Gossip 则在规模化场景下处理成员发现, 避免共识开销过重。
- **PostgreSQL 16、Redis 7 集群、etcd v3.5、SQLite**: 为不同任务选择合适的存储——Postgres 用于持久关系型数据, Redis 用于热缓存, etcd 用于协调, 嵌入式 SQLite 则作为节点本地 HXC 工作项注册表。
- **NATS 2.10 (JetStream)、Kafka 4.0 (KRaft)、RabbitMQ 3.13**: 三种消息骨干, 应对三类流量形态——NATS/JetStream 用于快速内部事件, Kafka 用于高吞吐持久流, RabbitMQ 则提供经典消息代理语义。
- **WireGuard 网状网 + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF**: WireGuard 构建精简的节点间网状网, 结合混合后量子握手及 AEAD 记录, 确保传输层对经典及量子攻击均具备保密性。
- **SPIFFE + JWT (HS256) + 基于范围的 RBAC + OPA**: 分层身份与授权——SPIFFE 提供工作负载身份, JWT 生成令牌, 基于范围的 RBAC 进行粗粒度访问控制, OPA 则将细粒度策略以代码形式表达。
- **Prometheus v2.50、Grafana 10.4、Jaeger 1.55、W3C 追踪**: 指标、仪表盘及分布式追踪, 采用 W3C 上下文传播, 使请求可跨服务及硬件层追踪。
- **HashiCorp Vault 1.16**: 密钥与凭证与代码及配置隔离, 并受审计发放。
- **Docker Compose、Kubernetes (kustomize, 强化 securityContext)、Helm**: Compose 用于本地启动, Kubernetes/Helm 结合强化安全上下文用于实际部署, 环境间共用一套定义。
- **React + TypeScript + Vite (Node 20+)**: 快速、类型安全的 Web UI, 用于会话、终端及资源池利用率展示。
- **TLA+**: 共识与调度不变量的形式化规约, 在实现前于设计层面证明最难测试的属性。

内容

状态与诚信说明

- **状态**: 开发中。当前版本为早期版本 (0.1.0-dev)。多项高级功能——包括全保密多节点推理全流程、市场结算机制及基于证明的调度填充——在代码库中明确标注为「计划中 / 基础设施受限」, 尚未作为完整可用功能呈现。覆盖率数据为自行报告。
- **许可证**: 待定。未明确声明; Helm 图表中的 HelixCluster/HelixCluster 及 helixcluster.io 链接均为未经验证的占位符, 与实际远程仓库不符。
- **捆绑的 LLM 栈项目 (LLMOrchestrator、LLMProvider、LLMsVerifier)** 为解耦子模块, 并非集群内托管的模型服务器。

优先级分级： Helix-主（LLM-基础设施集群，即可承载推理与计算工作负载的底层资源）。优先级低于 HelixTrack。